MAGI

Multivariate Autoencoder for particle Generative Inference

A CVAE-based generative particle source for Monte Carlo transport

User Manual — v0.8.2-beta

Francesco Monastra
INAF — `francesco.monastra@inaf.it`

August 1, 2026

MAGI v0.8.2-beta reproduces the *joint* phase-space distribution (energy–position–direction correlations) and the energy continuum to within its stated acceptance bars, on three seeds and two reference sources. It does **not** yet reproduce spectral-line *intensities*: measured per-line recovery runs from 0.7× to 4.9× the true value and is unstable across seeds. Line *positions* are reliable to ≤ 0.5 eV.
Read Section 8 before quoting any number derived from a MAGI-generated source. Do not use this release to estimate a fluorescence or annihilation line flux.

Contents

1 Introduction

1.1 What MAGI is

MAGI — *Multivariate Autoencoder for particle Generative Inference* — is a Conditional Variational Autoencoder (CVAE) [1, 2], implemented in Keras/TensorFlow, that learns the joint phase-space distribution of particles crossing a surface in a Monte Carlo simulation — their energy, position, direction and particle type — and then samples new, physically consistent events at a small fraction of the cost of running the full transport again.

Development and validation have been done against Geant4 [3, 4], and the shipped exporter writes Geant4-ready files, but nothing in the model is specific to it: MAGI learns from a table of crossing records and writes back a table in the same format it was given, so any consumer that can read that format can be driven from it.

The package is installed and imported as `magi`.

1.2 What it is for

The intended use is as a **drop-in replacement for a particle source** in a multi-stage simulation. Run the expensive full-physics simulation once to obtain a reference sample of particles crossing some surface; train MAGI on that sample; then use MAGI's output as the source for downstream simulations that would otherwise require repeating the expensive upstream physics.

This pays off when

- large numbers of particles are needed downstream;
- the events reaching the surface of interest are rare relative to the number of primaries thrown, so direct simulation is statistics-starved — whether the rarity is physical (a small interaction cross-section, a steep spectrum) or geometrical (a small solid angle, a deeply shielded volume);
- the same downstream simulation is run many times with different geometry or configuration choices, and re-deriving the upstream flux each time is wasteful.

Because MAGI's output schema matches its input schema, the natural way to use it is to cut a long simulation at a surface and resume from there — a decomposition that is useful in any transport problem with the statistics bottleneck above, not only the instrument-background case it was built for.

1.3 What it is not for

- Estimating the intensity of an individual spectral line (Section 8).
- Crossing surfaces that are not spheres. The surface MAGI is trained on is parametrized as a sphere (`center`/`radius`); see Section 9. Note this constrains the *scoring* surface, not the geometry being simulated: the surface is an intermediate bookkeeping step, so it is perfectly legitimate to place a dummy spherical tracking volume in an otherwise arbitrary mass model purely to give MAGI something to learn on. What is not supported is a non-spherical crossing surface.
- Extrapolating outside the energy range or particle mix of the training sample. MAGI is a density model of the data it was shown, not a physics engine.

1.4 How to read this manual

Section 3 describes the code and what each module is for. Section 4 is the API reference: functions, their settings and the contracts between them. Section 5 covers the command-line tools around a run. Sections 6 and 7 are worked examples of a full run and of a validation. Section 8 states what has actually been measured, and Section 9 lists the traps.

If you are new, start with `Example_Usage.ipynb` at the repository root: it is a runnable, commented walkthrough of the whole pipeline on synthetic data, and it finishes in a few minutes.

Energies are in MeV and positions in mm throughout, matching Geant4's own output convention. Code that must run on the CPU is shown with the `CUDA_VISIBLE_DEVICES=-1` prefix, which must be set *before* `magi` is imported.

2 Theoretical foundation

This section states what MAGI is built on and where each piece is implemented. It is background, not a derivation; the primary sources are cited so a reader can go to them directly.

The entries in `docs/manual/magi_theory.bib` are marked `% VERIFY`: their arXiv identifiers and titles are believed correct, but no author has yet opened each source to confirm author lists, venue, volume and pages. Confirm them on NASA ADS, INSPIRE or DBLP before any of this text is reused in a citable document.

2.1 Conditional variational autoencoders

A variational autoencoder [1] models a density $p(x)$ through a latent variable z, training an encoder $q(z|x)$ and decoder $p(x|z)$ against the evidence lower bound

$$\mathcal{L} \;=\; \mathbb{E}_{q(z|x)}[\log p(x \mid z)] \;-\; \mathrm{KL}(q(z \mid x) \,\|\, p(z))\,. \tag{1}$$

The conditional variant [2] conditions every term on an observed covariate c, giving $q(z|x,c)$, $p(x|z,c)$ and $p(z|c)$. In MAGI c is the particle-type one-hot, which is what lets generation be steered to a chosen particle mix rather than only reproducing the training proportions.

Implemented in `magi/core/model.py`; the reconstruction and KL terms in `magi/core/losses.py`.

2.2 Normalizing flows for the energy continuum

A normalizing flow [5, 6] represents a density as an invertible, differentiable map f applied to a simple base density, with the change-of-variables formula supplying an exact likelihood:

$$\log p(y) \;=\; \log p_u(f(y)) \;+\; \log \left| \det \frac{\partial f}{\partial y} \right|. \tag{2}$$

MAGI uses monotonic rational-quadratic spline transforms [7] for the energy continuum. The choice matters here for a specific reason: the crossing spectrum of a cosmic-ray source spans roughly nine decades and contains sharp non-Gaussian structure — a Compton edge, a muon peak, a steep low-energy tail — that a single Gaussian cannot represent at all, and that an affine flow represents only poorly. A spline flow is flexible enough to follow that structure while remaining exactly invertible, which the exact-likelihood training requires.

Implemented in `magi/core/flows.py` (`ConditionalRQSFlow`), selected with `continuum_mode="flow"`.

2.3 The gated line–continuum mixture

The central modelling decision in v0.8 is that spectral line positions are *not learned*. A categorical energy head can never place a line more precisely than one bin, and on a log grid spanning nine decades a bin is hundreds of eV — orders of magnitude wider than a real fluorescence line. Positions therefore become fixed physical inputs, measured from the data and taken from evaluated atomic data, and the head becomes a mixture:

$$p(y \mid z, c) \;=\; g_0(z,c)\, p_{\text{cont}}(y \mid z, c) \;+\; \sum_{\ell=1}^{L} g_\ell(z,c)\, \mathcal{N}\!\left(y\,;\, y_\ell,\, \sigma_\ell^2\right), \qquad \sum_{\ell=0}^{L} g_\ell = 1, \tag{3}$$

with $y = \log_{10}(E/\text{MeV})$. The widths σ_ℓ are pinned to the instrument's energy resolution rather than fitted.

The gate g faces a severe class imbalance: a line can be 10^{-5} of the sample, so an unweighted cross-entropy is minimized well by never routing anything to it. MAGI therefore applies focal down-weighting of easy examples [8], exposed as `gate_focal_gamma`.

Use $\gamma = 1$. At $\gamma \geq 2$ the gate over-corrects and begins routing genuine continuum events into the line components, visibly digging a hole in the continuum on either side of each line. This is measured, not assumed: the near-line continuum ratio degrades monotonically across a γ sweep.

2.4 The learned conditional prior

With a standard $p(z) = \mathcal{N}(0, I)$, two problems arise. First, the decoder alone must carry every correlation between energy and geometry. Second, and more subtly, there is a **train/generate mismatch**. During training the decoder only ever sees latent codes produced by the encoder from a real event, $z \sim q(z \mid x)$. At generation time there is no real event to encode, so codes are drawn from the prior instead, $z \sim p(z)$. These two distributions are the same only if the *aggregated* posterior $q(z) = \mathbb{E}_x\, q(z \mid x)$ — the pooled cloud of codes over the whole dataset — happens to coincide with the prior. When it does not, the generator samples regions of latent space the decoder was never trained on and produces events from an extrapolation [9].

The `kl`/`val_kl` numbers in the training log are the ELBO's own regularization term, $\mathrm{KL}(q(z \mid x) \,\|\, p(z \mid c))$, averaged over events. They measure the encoder against the prior in *latent* space only, and say nothing directly about the generated energies, positions or directions — that is what Section 7 measures.

On the reference sources this sits at 21–28 nats with `latent_dim` $= 8$, i.e. roughly 3 nats per latent dimension. Two things are worth reading off it, and one is not:

- **It is not collapse.** The classic VAE failure is KL $\to 0$: the encoder ignores the input and the decoder ignores z. Nothing near that is happening here.
- **Training and validation KL track each other** (e.g. 22.30 versus 22.27 at the last epoch of one CR run), so the latent code is encoding structure that generalizes rather than memorizing the training split.
- **It is not a quality score, and bigger is not worse.** With $\beta = 0.2$ fixed and no annealing, KL rises through training precisely *because* reconstruction keeps improving — the usual rate–distortion trade-off. A run that ends at a higher KL than another is not thereby a worse run.

What the number does justify is the design decision: a gap of this size between $q(z)$ and a fixed $\mathcal{N}(0, I)$ is exactly the regime where the train/generate mismatch above bites, which is why the prior is learned rather than fixed. Note that once `prior="coupling"` is in use the reported KL is measured against that *learned* prior, so it is no longer the "distance from $\mathcal{N}(0, I)$" quantity the argument started from, and should not be compared across runs that used different priors.

MAGI replaces the fixed prior with a learned conditional coupling flow $p(z \mid c)$, built from affine coupling layers [10] and trained with a Monte-Carlo estimate of the KL term. This is

what brings the coupling residuals of Section 8 inside their bar, and it is the difference the v0.7.2 categorical head cannot make up.

In v0.8.2 the prior's conditioning is widened from the particle-type one-hot alone to $[\,c,\,\text{zone}\,]$, where zone is the continuum/line routing target (`prior_zone_conditioning=True`). Without the zone in the conditioning, the prior has no way to express which mixture component an event belongs to, so at generation time rare lines are routed by whatever the type marginal happens to imply.

Implemented in `magi/core/priors.py` (`ConditionalCouplingPrior`).

2.5 Atomic transition energies

Line positions are only as good as the table they are pinned at. MAGI takes transition *labels* (Kα_1, Kβ, and so on) from Bearden's compilation [11], but transition *energies* from the EADL evaluated library [12], because EADL is what the Geant4 physics list actually emitted.

This distinction is not pedantic. Pinning to Bearden energies while the simulation emits EADL displaces Cu Kα_1 by 42 eV and Cu Kβ by 40 eV — around ten times the 4 eV instrument FWHM the generated lines are given. The generated and real lines then have essentially no overlap, and no amount of gate tuning can recover it. Every pinned position is audited against a centroid measured from the real spectrum (`tools/line_centroid_audit.py`).

2.6 Optimization

Training uses Adam [13]. The per-variable loss terms are combined with fixed weights (`w_energy`, `w_ur`, ..., `w_gate_aux`) chosen so that no single head dominates the objective purely because its natural scale is larger.

A task-adaptive loss-weight scheduler that rebalances those weights *during* training exists in `magi/training/adaptive_callbacks.py` and gives the `..._TaskAdaptive` classes their name, but it is opt-in and was not used for any v0.8 result reported here — see the warning in Section 4.

2.7 Relation to other work

MAGI's problem — learn a compact, resampleable particle source on a surface so that a multi-stage Monte Carlo can be decoupled — is the same one KDSource [14] addresses with kernel density estimation. MAGI substitutes a deep conditional generative model, which scales better to the joint energy–position–direction–type density and conditions naturally on particle type.

The broader use of deep generative models to replace expensive detector simulation is well established in high-energy physics [15], with GAN- [16] and flow-based [17] calorimeter shower models now in production use [18]. MAGI differs in target: rather than showers inside a calorimeter, it models the particle flux crossing a surface, so that a transport problem can be cut at that surface and resumed from a cheap sampler.

That target is not tied to one instrument or one energy regime. The conditions MAGI is built for are structural: the events that matter are rare at the surface of interest — because the cross-section is small, the spectrum is steep, the solid angle is small, or the volume is deeply shielded — and the problem admits a multi-stage decomposition, so the expensive upstream physics can be paid for once and resampled many times. Any simulation with that shape is a candidate. The development and validation reported here were carried out on the cryogenic X-ray instrument background problem [19], which supplies this manual's two reference sources, but the numbers in Section 8 are properties of those sources rather than limits of the method — measure on your own source with the acceptance harness rather than assuming they transfer.

3 Structure of the code

3.1 Repository layout

The repository has two parts: the installable library, and the experiment material that uses it.

Path	Purpose
`MAGI_package/`	The installable Python library (`import magi`). Everything in Section 4 lives here.
`Example_Usage.ipynb`	Runnable walkthrough on synthetic data. Start here.
`MAGI_v0_*.ipynb`	The experiment notebooks. Highest version number is the current pipeline; `OldNotebooks/` holds deprecated ones.
`tools/`	Command-line scripts *around* a run rather than inside one: full training runs, the acceptance harness, audits, synthetic stress tests (Section 5).
`scripts/generate_geant_source.py`	Produces a Geant4-ready source file from a trained run, outside a notebook. This is what the companion Geant4 project's `/generator/mlScript` macro invokes (Section 6.3).
`CandidateLines/`	Candidate spectral-line tables built from a GDML mass model, in the JSON form `magi.load_candidate_energy_lines` reads.
`TrainingData/`	Cleaned, whitespace-delimited crossing data. Gitignored — large binaries stay local.
`trained_models/`, `checkpoints/`	Trained-run artifacts. Weights are gitignored; the JSON config/metadata is versioned.
`docs/`	Development notes, measurement write-ups and plans for the v0.8 line, plus this manual under `docs/manual/`.

3.2 Package layout

```
magi/
  __init__.py       public API surface (the canonical list of exports)
  magi.py           high-level convenience API: setup, build_model, train_model
  config.py         environment/seed initialization
  core/
    model.py        the CVAE classes, one per version in the lineage
    losses.py       shared NLL / angular / mixture losses
    geometry.py     physical coordinate transforms
    flows.py        conditional rational-quadratic-spline flow (v0.8 continuum)
    priors.py       conditional coupling prior p(z|cond)   (v0.8)
  data/
    io.py           load/save raw detector tables, normalization, line tables
    preprocessing.py physical features, energy binning, line detection,
                     quantile transforms, gate targets
    dataset.py      split, scale, one-hot conditioning, tf.data construction
  training/
    train.py            compile/fit wrappers
    adaptive_callbacks.py task-adaptive loss rebalancing and monitors
    checkpointing.py    save/load weights + metadata + transformers as a unit
  generation/
    sampling.py       draw latent codes and conditioning
    reconstruction.py invert model outputs back to physical quantities
    export.py         write Geant4 source files (text or chunked binary)
  validation/
    metrics.py      Wasserstein, line-integral recovery
    compare.py      histogram-residual comparisons, range/norm checks
  utils/
    plotting.py         all plot_* helpers, light/dark theme
    model_inspection.py introspect a built model
```

```
    circuit_viz.py       interactive routing-circuit HTML (Section~\ref{sec:visualization})
    full_circuit.py      interactive per-event gradient-circuit HTML (Section~\ref{sec:
    visualization})
tests/              142 tests; see Section~\ref{sec:tools}
docs/USAGE.md       short-form user guide
```

3.3 The data flow

Every run follows the same seven stages. Each arrow is a function whose output is the next one's input; each stage has a matching `report_*` or `print_*` helper that prints sanity checks and returns nothing.

1. **Load** — `load_detector_table` reads the raw crossing file into a DataFrame.
2. **Physical features** — `build_physical_features` turns Cartesian position/direction into the sphere-surface parametrization $(u_r, \phi_r, u_v, \phi_v)$ and measures the primary fraction.
3. **Feature frame** — `build_feature_dataframe` bins energy, detects spectral lines and fits the quantile geometry transforms.
4. **Dataset** — a five-step chain, each feeding the next:

   `filter_particle_types_continuous_geometry` → `split_feature_data` → `scale_continuous_features` → `build_conditioning_and_weights` → `build_tf_datasets`.

5. **Train** — a class from `magi.core`, then `compile_model` and `fit_model`.
6. **Checkpoint** — `save_final_trained_model` writes weights, config, metadata, history, task weights and a summary as one unit.
7. **Generate and validate** — `generate_latent_outputs` → `reconstruct_generated_features` → `reconstruct_generated_physics` → export, with `magi.validation` scoring the result.

3.4 Why per-variable heads

MAGI does not predict one flat output vector. Each physical variable gets its own head, because each has a different pathology:

- u_v is bounded on $[-1, 1]$ and piles up at the edges. A Gaussian head fits it badly, so the model works in $s = \operatorname{atanh}(u)$ space, or on a quantile transform of it.
- ϕ_r, ϕ_v are periodic. The v0.7 heads predict a $(\cos, \sin)$ pair softly regularized onto the unit circle; v0.7.2 predicts the angle through a quantile transform directly.
- Energy spans many decades and contains narrow spectral lines. This is the axis the version lineage is really about (Section 3.5).

3.5 Model variants

The classes in `magi.core` form a lineage. A trained run records which one it is in `model_config["model_class"]`; check that before assuming behaviour.

Class	Version	Energy / geometry
`CVAE_CatEnergy_CatUV`	v0.6	Categorical energy, discretized u_v.
`CVAE_CatEnergy_CatUV_TaskAdaptive`	v0.6	As above, plus task-adaptive loss weighting.
`CVAE_CatEnergy_ContGeom_TaskAdaptive`	v0.7	Continuous geometry, $(\cos, \sin)$ angles.
`CVAE_CatEnergy_ContPhi_TaskAdaptive`	v0.7.2	Continuous ϕ_r, ϕ_v. The stable fallback head.
`CVAE_MixEnergy_ContPhi_TaskAdaptive`	v0.8	v0.7.2 geometry; energy becomes a gated mixture of a continuum density and fixed-position line components.

The two changes that define v0.8 are described in the theory section rather than repeated here: the **gated line–continuum mixture** that replaces the categorical energy head (§2, "The gated line–continuum mixture"), and the **learned conditional prior** that replaces $\mathcal{N}(0, I)$ (§2, "The learned conditional prior"). Both are selected through constructor arguments — `continuum_mode`, `prior` — documented with the rest of the training settings in Section 4.

Use `magi.print_model_structure(model)` to print a built model's generative structure, formulas, configured line table and parameter counts.

4 Usage: functions and settings

Every exported function carries a full NumPy-style docstring, so hovering over a name in an editor shows its parameters and return value. This section covers the contracts *between* functions, which a per-function docstring cannot.

This is an editor configuration problem, not a missing docstring. `pip install -e` installs `magi` through a PEP 660 editable hook: setuptools writes an import *hook* (`site-packages/__editable___magi_*_finder.py`) rather than a path entry, and Pylance's static analysis cannot follow it. The code runs perfectly; the editor simply cannot find the source, so every `magi` symbol silently resolves to nothing.
The repository now ships `.vscode/settings.json` pointing the analyzer at the source tree directly, which fixes it regardless of how the package was installed:

```
{"python.analysis.extraPaths": ["MAGI_package"]}
```

If you work from a different editor or workspace root, set the equivalent option to `MAGI_package/`. Reloading the window is required for the analyzer to pick it up.

4.1 Installation

```
pip install -e MAGI_package/      # develop the library
pip install -r requirements.txt   # or install it as a dependency
```

Core dependencies: `tensorflow` (plus `tensorflow-metal` on Apple Silicon), `tensorflow-probability`, `scikit-learn`, `joblib`, `astropy`, `optuna`.

4.2 Environment setup

```
import magi
magi.initialize_environment(seed=42, cpu_only=True, quiet=True)
magi.print_tf_info()
```

`cpu_only=True` only takes effect if it runs before TensorFlow initializes its devices. In a script that imports `magi` at module level this is already too late — set the environment variable instead, before any import:

```
import os
os.environ["CUDA_VISIBLE_DEVICES"] = "-1"
import magi
```

Every script in `tools/` does exactly this, and for a reason: on this workload `tensorflow-metal` is slower than the CPU, and importing `tensorflow_probability` initializes the Metal device,

after which `set_visible_devices` silently stops working.

4.3 Loading data

```
df = magi.load_detector_table("TrainingData/alloutputDSCryoSphereCR.dat")
magi.report_basic_table_checks(df)
```

Three column schemas are auto-detected from the field count on the first data line. Each is a strict prefix of the next, so the columns always appear in the same order and a wider file is readable by anything that understands a narrower one:

Cols	Schema	Column order, left to right
9	legacy	`EventId ParticleName Energy X Y Z Vx Vy Vz`
10	PrimBool-tagged	*the 9 above*, then `PrimBool`
13	lineage	*the 10 above*, then `ParticleId ParentParticleId CreatorProcessName`

Column	Meaning and units
`EventId`	Geant4 event number the crossing belongs to.
`ParticleName`	Geant4 particle name (`gamma`, `e-`, `mu-`, ...). Becomes the conditioning one-hot.
`Energy`	Kinetic energy at the crossing, **MeV**.
`X Y Z`	Crossing position, **mm**, in the mass model's frame.
`Vx Vy Vz`	Direction, a normalized Cartesian unit vector.
`PrimBool`	1 if the particle is a Geant4 primary (`ParentID == 0`). Used for the cosmic-ray primary fraction.
`ParticleId`	Track ID of this particle.
`ParentParticleId`	Track ID of its parent, 0 for a primary.
`CreatorProcessName`	Process that created it (`RadioactiveDecay`, `compt`, ...). This is the column that makes the radioactive primary fraction computable.

Choose the widest schema your simulation can write. The 13-column lineage form is **required** for radioactive sources — see the `source_type` paragraph below for why `PrimBool` alone cannot answer the question there.

4.4 Preprocessing

```
prep = magi.build_physical_features(
    df, center=(0.0, 0.0, -507.66), radius=100.0,
    source_type="cosmic_ray",
)
magi.print_physical_summary(prep)

feature_pack = magi.build_feature_dataframe(
    prep,
    energy_binning_mode="log_fixed_count",
    n_bins=512,
    min_counts=20,
    geometry_transform="quantile_u_r_u_v_phi_r_phi_v",
    energy_transform="log10",      # required by the v0.8 mixture head
)
magi.report_feature_dataframe(feature_pack)
```

`source_type` decides flux normalization. With `"cosmic_ray"` a crossing counts as primary iff `PrimBool == 1`. With `"radioactive"` it counts as primary iff `CreatorProcessName== "RadioactiveDecay"`, because for a source embedded in bulk material the literal Geant4 primary is the decaying nucleus, which never propagates — `PrimBool` is always 0. Choosing wrongly does not error; it silently yields a wrong normalization factor.

A fair question, since `energy_transform="log10"` means the value the network actually sees is $y = \log_{10}(E/\mathrm{MeV})$, a *continuous* scalar. Nothing rounds it to a bin. With the v0.8 mixture head the binning has stopped being part of the model input at all; what survives is an **analysis grid**, used by three things that genuinely need a histogram:

- **Line detection.** `detect_energy_lines` finds peaks by prominence, which requires counts in bins. You cannot run a peak finder on an unbinned list of floats.
- **Gate targets.** `build_gate_targets` evaluates each line's membership kernel on this grid, and its `"bins"` bandwidth mode is expressed as a multiple of the local bin width.
- **Validation.** `compute_line_integral_recovery` counts real and generated events in the same windows, and `compare_hist_with_residuals` plots on them.

So in v0.7.2, where the energy head was categorical, the bin width was a hard floor on line placement. In v0.8 it is not: line positions are pinned physical inputs and the continuum is a continuous density. The binning now limits which lines can be *found*, never where a found line can be *put*.

These are two different passes with two different jobs, and they use different grids on purpose. The `log_fixed_count` grid is for **detection**. A cosmic-ray crossing spectrum spans about nine decades, so no single fixed width works: a width fine enough to resolve a 1.5 keV line puts essentially zero counts per bin at 100 GeV, and a width coarse enough to be populated up there is hundreds of keV wide down low. `log_fixed_count` instead adapts the width to hold roughly constant counts per bin across the whole range, which is what lets one prominence threshold work at both ends.

The fixed-resolution numbers are for **measurement**, in a second, local pass: `refine_bin_width_mev` re-histograms a narrow neighbourhood of each candidate at the instrument's own scale (4 eV here), and `confirm_unresolved_candidate_lines` does the same for candidates the coarse pass could not separate. `tools/line_centroid_audit.py` then verifies the outcome against the real spectrum.

Coarse adaptive grid to find peaks; fine fixed grid to place them. Using the detection grid for placement is the v0.7.2 error the mixture head exists to remove.

`geometry_transform` is a stringly-typed contract. The value (`"arctanh_uv_discrete"`, `"quantile_u_r_u_v"`, `"quantile_u_r_u_v_phi_r_phi_v"`) fixes the column ordering shared by preprocessing, the model's input layout and reconstruction. It is persisted in the saved metadata and **must** match between the run that trained a model and the run that generates from it. A mismatch does not raise — it produces physically wrong output.

4.5 Spectral lines

Line handling is the part of the pipeline most specific to v0.8, and the part where a silent mistake is most expensive.

Note the ordering: lines are detected on the *raw* energies, on their own fine binning, *before* `build_feature_dataframe` is called. The feature frame's binning is a separate, coarser thing —

with the mixture head it no longer constrains where a line can be placed, only how the continuum is described. The resulting gate targets are then carried into the dataset as extra feature columns.

```
payload = magi.load_candidate_energy_lines("CandidateLines/cryosphere_lines.json")
candidate_lines = payload["lines"]

res = magi.detect_energy_lines(
    E, binning_mode="log_fixed_count", n_bins=1024,
    candidate_lines=candidate_lines,
    refine_bin_width_mev=4.0e-6)   # refine to the pinned line width
magi.print_detected_energy_lines(res)

matched = [m for m in res["matched_lines"] if m["count"] >= 100]
matched += magi.confirm_unresolved_candidate_lines(
    E, candidate_lines, matched, resolution_ev=4.0)

gate_targets = magi.build_gate_targets(
    E, feature_pack["energy_bins"], matched,
    bandwidth_mode="resolution",       # the default; see the warning below
    bandwidth_fwhm_mev=4.0e-6)
line_logsigma = magi.line_logsigma_from_resolution(
    [m["candidate_energy_mev"] for m in matched], resolution_ev=4.0, fwhm=True)
```

`bandwidth_mode` controls how `build_gate_targets` decides which *real* events count as line events when building the gate's training target. It does not affect the generated line width, which is pinned separately via `line_logsigma_from_resolution`.

Raw Geant4 crossing data has **no detector response applied**, so simulated fluorescence lines are exactly monoenergetic: in the reference cosmic-ray source all 7 542 Cu Kα_1 events share one identical `float64` energy, and every nearby continuum event is a singleton. On that reasoning, using the instrument's 4 eV FWHM as a Gaussian kernel to decide line membership is the wrong tool — it hands weight to continuum a few eV away and blurs doublets closer than a few resolution widths — and `bandwidth_mode="exact"` was added to match the simulation's own ground truth.

Measured on the cosmic-ray source, it made things worse. It did not fix the Cu Kα_1 overshoot it was built for (2.011 versus 2.052, inside the noise), it regressed the previously passing Al Kα_1 from 0.959 ± 0.025 to 0.578, it regressed Cu Kβ from 1.034 ± 0.050 to 0.753, and it pushed the coupling residual from 0.0285 ± 0.0101 to 0.054, outside its bar. The tight tolerance also drove Cu Kα_2's zone probability to zero, effectively deleting it as a modelled component.

`"resolution"` is therefore the default and is what every confirmed result in Section 8 was produced with. `"exact"` remains available and tested, but it is a research option, not a recommendation.

The lesson is worth stating plainly: a training *label* that is more faithful to the simulation is not automatically a label that trains better. The resolution kernel appears to be doing useful work as a soft target, and replacing it with a hard one removed that.

`confirm_unresolved_candidate_lines` exists because a doublet closer together than one detection bin is always a single peak, so only one member is ever detected regardless of how good the matching logic is — the gap is in peak detection, not matching. It re-measures every unmatched candidate at eV resolution and confirms the ones that are real. Both `tools/run_v0_8_real.py` and `tools/acceptance_v0_8.py` must call it identically, or the checkpoint's `n_lines` silently misaligns against the rebuilt pipeline's line count.

4.6 Building the dataset

```
dataset_pack   = magi.filter_particle_types_continuous_geometry(
                     feature_pack["feat"], prob_threshold=1e-5)
split_pack     = magi.split_feature_data(dataset_pack, random_state=42)
scaled_pack    = magi.scale_continuous_features(split_pack, scale_cols=())
condition_pack = magi.build_conditioning_and_weights(
                     scaled_pack, dataset_pack["n_types"], alpha=0.5)
ds_pack        = magi.build_tf_datasets(condition_pack, batch_size=4096)
magi.report_tf_datasets(ds_pack)
```

Note `scale_cols=()` for v0.7+ quantile geometry: those columns are already standardized by their own transform, and scaling them twice is a mistake the empty tuple prevents. The split is stratified on particle type; `alpha` controls how strongly rare species are up-weighted (0 = natural proportions, 1 = fully balanced, 0.5 = default).

4.7 Training

```
model = magi.CVAE_MixEnergy_ContPhi_TaskAdaptive(
    n_types=dataset_pack["n_types"], latent_dim=8,
    line_positions_y=line_positions_y,
    line_logsigma_init=line_logsigma,
    continuum_mode="flow",
    continuum_flow_warp="cdf",
    energy_flow_condition="z_cond",
    prior="coupling",
    prior_zone_conditioning=True,
    zone_probs=zone_probs,
    gate_focal_gamma=1.0,
    w_gate_aux=2.0,
)

magi.compile_model(model, learning_rate=2e-4, optimizer="adam")
history = magi.fit_model(
    model, ds_pack["train_ds"], ds_pack["val_ds"],
    epochs=40, callbacks=magi.build_default_callbacks(), verbose=2)
```

The constructor takes `n_types` and `line_positions_y` positionally; there is no `n_cont` argument. The number of continuous targets is not configurable — it is fixed by the geometry parametrization at four (u_r, u_v, ϕ_r, ϕ_v) plus energy plus the $L + 1$ gate-target columns, and the model derives it from `line_positions_y`.

Setting	Default	Effect
`latent_dim`	8	Latent width. Larger widens the posterior/prior gap that the coupling prior exists to close.
`continuum_mode`	`"flow"`	A single Gaussian cannot represent a multi-decade continuum. `"gaussian"` is kept for regression only.
`continuum_flow_warp`	`"affine"`	**Set this to `"cdf"` yourself.** It pre-warps the flow's input by the empirical CDF, which is what recovers the Compton edge and the low-energy tail. It is the one recommended value that cannot be a default: it requires `continuum_flow_warp_y_knots/_z_knots`, which are fit from your training energies with `magi.fit_cdf_warp_knots`, and the constructor never sees the data. Passing `"cdf"` without knots raises.
`energy_flow_condition`	`"z_cond"`	Conditions the continuum flow on both latent and conditioning.
`prior`	`"coupling"`	The learned conditional prior (§2). This is what makes coupling residuals pass.
`prior_zone_conditioning`	`False`	Condition the prior on `[type, zone]` rather than type alone. Requires `zone_probs`.
`gate_focal_gamma`	1.0	Focal weighting on the gate loss. Keep at 1; $\gamma \geq 2$ digs the continuum out from between the lines.
`w_gate_aux`	0.3	Weight of the auxiliary gate-routing loss. The reference runs use 2.0.

`continuum_mode`, `prior` and `gate_focal_gamma` previously defaulted to `"gaussian"`, `"gaussian"` and `0.0` — the v0.7.2-era behaviour. They now default to the validated v0.8 configuration, so a bare constructor call builds the recommended model rather than the legacy one.
Checkpoints are unaffected. The loader requires every architecture key explicitly (Section 4, checkpointing) and a saved config always records the values the run actually used, so a pre-flip checkpoint still reconstructs its original architecture — there is a regression test asserting exactly that. What *does* change is any script that constructed the model without naming these arguments and relied on the old defaults; every such call site in this repository has been pinned to its previous explicit value.

Despite the class name `..._TaskAdaptive`, `build_default_callbacks` returns only `EarlyStopping` (restoring best weights) and `ReduceLROnPlateau`. It deliberately does **not** include `TaskAdaptiveLossScheduler` from `magi/training/adaptive_callbacks.py`, and neither `tools/run_v0_8_real.py` nor any v0.8 result in this manual used it — the per-task loss weights stayed at the fixed values passed to the constructor (`w_energy`, `w_ur`, ..., `w_gate_aux`). The scheduler was used in the v0.7/v0.7.2 line and remains available; add it to the callback list yourself if you want it. Do not assume it is running because the class name says "TaskAdaptive".

4.8 Checkpointing

```
magi.save_final_trained_model(
    model=model,
    save_dir="trained_models/my_run",
    model_name="mix_CR",
    history=history,
    model_config=model.to_generation_config(),
    preprocessing_metadata=preprocessing_metadata,
)
joblib.dump(transformers, "trained_models/my_run/mix_CR_quantile_transformers.joblib")
```

A run directory holds `*.weights.h5`, `*_config.json`, `*_metadata.json`, `*_history.json`, `*_task_weights.json`, `*_summary.txt` and — for quantile-geometry runs — `*_quantile_transformers.joblib`. Generation needs the config and metadata as much as the weights, because they carry the transforms that invert the preprocessing. Copy or move them together.

Always pass `model.to_generation_config()` rather than a hand-written dict. For v0.8 heads the loader *requires* every key it emits and raises on a missing one. This guard exists because the old behaviour was `model_config.get(key, default)` everywhere, and the defaults (`continuum_mode="gaussian"`, `prior="gaussian"`, `continuum_flow_warp="affine"`) select a *code path* rather than a layer shape — so a renamed key produced a silently different architecture that loaded without complaint and generated wrong output.

Checkpoints from v0.8.2 carry `config_version: 2`. Older `config_version: 1` checkpoints load unchanged: `prior_zone_conditioning` defaults to `False`, reconstructing the exact pre-existing architecture.

4.9 Generation

```
model = magi.load_task_adaptive_model_for_generation(
    save_dir="trained_models/my_run", model_name="mix_CR",
    model_config=magi.load_json("trained_models/my_run/mix_CR_config.json"),
    n_types=n_types, type_weights=type_weights, radius=100.0)

gen_pack = magi.generate_latent_outputs(
    model, n_samples=n_real, type_probs=type_probs,
    n_types=n_types, idx_to_type=idx_to_type)

reco = magi.reconstruct_generated_features(
    gen_pack, energy_head_mode="mixture",
    geometry_metadata=geometry_metadata, energy_metadata=energy_metadata)

gen = magi.reconstruct_generated_physics(
    reco, center=(0.0, 0.0, -507.66), radius=100.0)
```

The line-recovery metrics compare raw counts. Generating fewer events than the real sample deflates every ratio by exactly the shortfall — a 3.44× error on the reference cosmic-ray source. Set `n_samples` to the real event count when validating.

For a Geant4-ready file, use the one-call entry point, which loads the checkpoint and streams events to disk in chunks:

```
magi.generate_detector_input_file(
    save_dir="trained_models/my_run", model_name="mix_CR",
    model_config=model_config, n_types=n_types,
    type_weights=type_weights, type_probs=type_probs,
    idx_to_type=idx_to_type, geometry_metadata=geometry_metadata,
    energy_head_mode="mixture", energy_metadata=energy_metadata,
    output_file="generated/source.dat", n_events=10_000_000,
    radius=100.0, center=(0.0, 0.0, -507.66),
    output_format="binary", seed=42)
```

Neutrinos are filtered out by default (they do not contribute to detector background here). With `generate_until_n_written=True`, the default, generation continues until `n_events` *transportable* particles have been written, so Geant4 receives exactly the requested count.

4.10 Visualization

Two self-contained, interactive HTML tools in `magi.utils` inspect a trained v0.8 mixture run without a plotting library or a running kernel — each call writes one `.html` file that opens in any browser, with its own light/dark palette and no external dependency.

`save_routing_circuit` The gate's conditional routing, per type: for each particle type, a fan-out diagram of one-hot conditioning → gate → zone, colored by the checkpoint's own `zone_probs` — the same per-type empirical routing frequency the learned conditional prior is conditioned on (§2).

```
magi.save_routing_circuit(save_dir="trained_models/v0_8_2_priorzone_CR",
                          model_name="mix_CR")
```

Reads only the checkpoint's own config/metadata — no raw data, no loaded model, so it runs in under a second. Requires `prior_zone_conditioning=True` at training time: a `config_version 1` checkpoint (or any run with zone-conditioning off) has no `zone_probs` to plot and raises `KeyError`.

`save_full_circuit` How heavily every unit in the network is actually *used*, per particle type, aggregated over many real held-out events (Figure 2).

The metric is **Expected Conductance**. Integrated Gradients [20] attributes a prediction to *input features*; conductance [21] is its generalization to *internal units*, formally path-integrated gradients evaluated at a hidden layer, at comparable cost. Rather than a single arbitrary baseline, each event is attributed against a pool of other real held-out events of the same type and averaged, following expected gradients [22]. A zero baseline would be meaningless here: inputs are quantile-transformed, so 0 is a real, specific quantile rather than the absence of a signal.

Two details make the quantity well defined. Only the continuous feature block is interpolated along the attribution path — the one-hot type conditioning is held fixed, since linearly interpolating a categorical is not meaningful — and the attribution target is the model's own per-event reconstruction loss, not just the gate logit, so branches with no gradient path to the gate (the energy branch reads the decoder stem directly, bypassing the deep trunk that feeds position and direction) still get genuine, nonzero attribution.

```
magi.save_full_circuit(
    save_dir="trained_models/v0_8_2_priorzone_CR", model_name="mix_CR",
    training_data_filepath="TrainingData/alloutputDSCryoSphereCR.dat",
    candidate_lines_file="CandidateLines/CANDIDATE_ENERGY_LINES_"
                         "SRON_CCNwithXFDM_NoShield_FlowerCryoAC_fixed_EADL.json",
    center=(0.0, 0.0, -507.66), radius=100.0,
    n_events_per_type=128, n_baselines=16, n_steps=32)
```

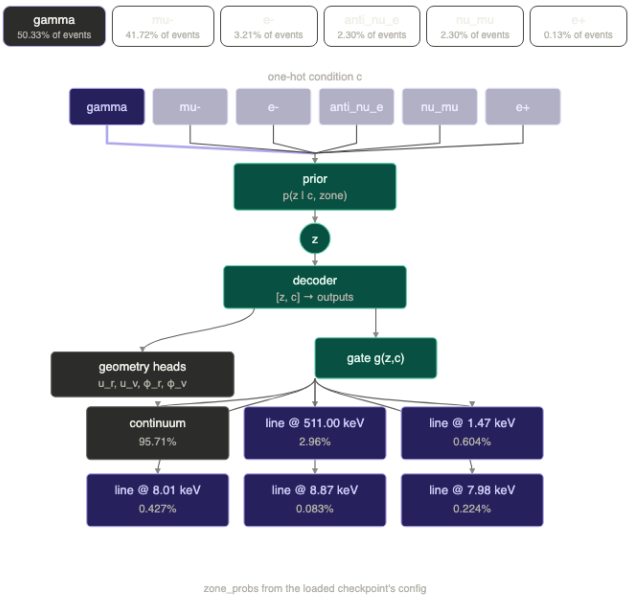

Figure 1: `save_routing_circuit` on the reference CR run. Selecting a particle type highlights its one-hot wire and fills in that type's routing percentage on every zone leaf. Here `gamma` sends 95.7 % of its events to the continuum and the rest to the five pinned lines; `mu-` routes essentially everything to the continuum, which is the physically correct answer and a useful sanity check that the gate has not learned to sprinkle lines across types that cannot produce them.

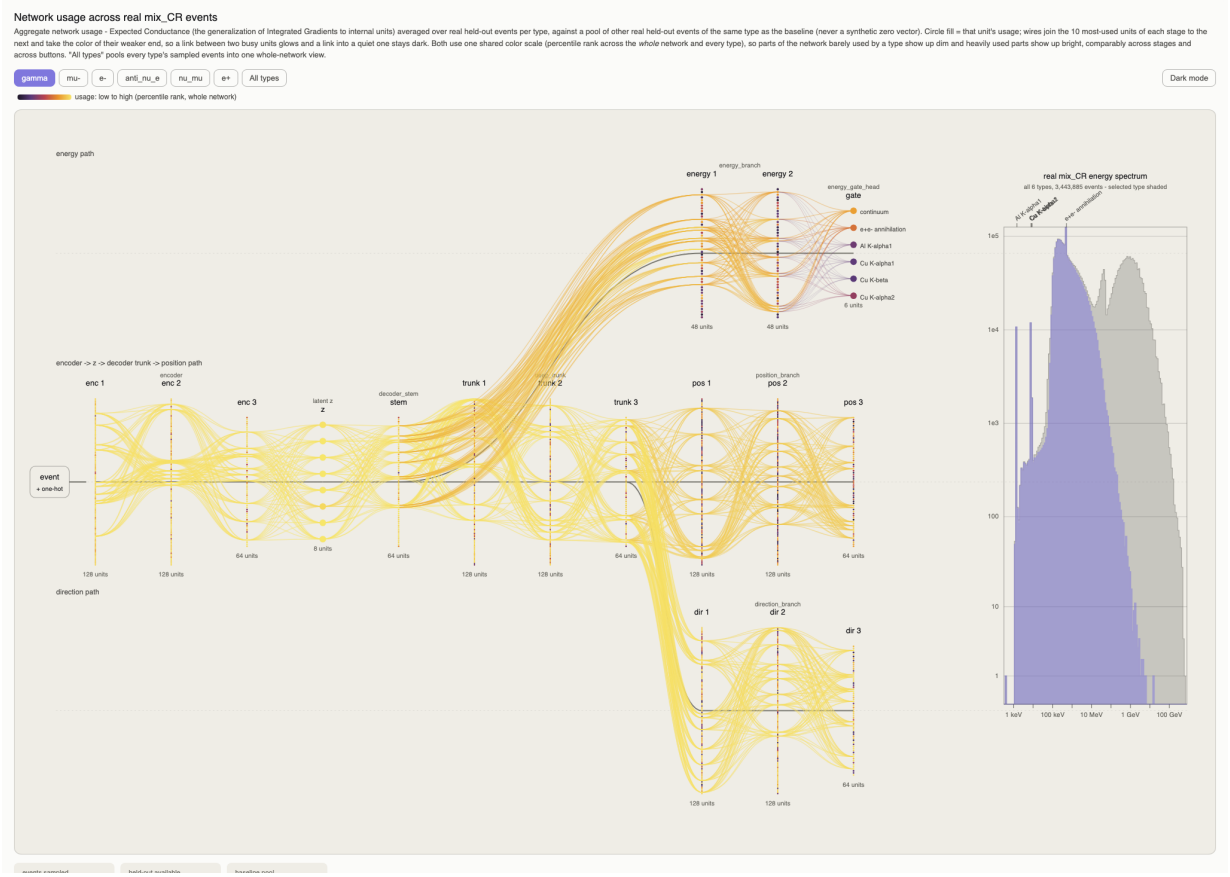

Figure 2: `save_full_circuit` on the reference CR run, with `gamma` selected (the view the file opens on). Each column is one layer, each dot one unit colored by usage; wires join the ten most-used units of each stage to the next and take the color of their weaker end. The encoder, latent z and decoder stem run bright, and the energy path fans up into the gate, whose six zones are labelled at the right against the real CR spectrum. Switching to "All types" pools every type's events into one view — that is the grouping Table 1 reports.

Reading it: what the colors do and do not say

Usage is heavy-tailed — most units contribute almost nothing and a few dominate — so a linear or even log color scale leaves roughly three quarters of every layer in the colormap's near-black lower third and the whole diagram reads as uniformly dark. The colors are therefore *percentile ranks*, pooled over all types so the buttons stay comparable with each other.
The consequence matters: **by construction some units always look dark**, whatever the true distribution is. A dark unit is low-ranked, not unused. Do not read the colors as evidence of spare capacity.

For that question, open the **per-layer absolute usage** panel below the diagram (Table 1), which reports the unnormalized magnitudes. The column to read is `load`: a layer's share of the network's total conductance divided by its share of the network's units. 1.0× means the layer pulls exactly its weight; 0.2× means it holds five times more units than its contribution justifies.

Layer	units	share	load	top-20 %	faint
`enc1-enc3`	128/128/64	10.3–10.5 %	1.17–2.39×	39–46 %	0–3 %
`z`	8	7.8 %	14.14×	53 %	0 %
`stem`	64	11.4 %	2.59×	56 %	2 %
`trunk1-3`	128/128/64	6.7–9.2 %	0.93–1.53×	51–57 %	7–14 %
`eb2`	48	1.4 %	**0.42**×	70 %	23 %
`pos1`	128	2.1 %	**0.24**×	60 %	16 %
`pos2`	128	1.9 %	**0.22**×	63 %	20 %
`pos3`	64	1.6 %	**0.37**×	57 %	17 %
`dir1-3`	128/128/64	5.1–5.9 %	0.64–1.15×	52–58 %	14–22 %

Table 1: Per-layer absolute usage, CR reference run, all types pooled. Shaded rows are the ones the tool flags: `load` below 0.5× *and* a real population of faint units. `position_branch` holds 320 units — 22 % of the network's width — and does 5.6 % of its work, while `direction_branch` at identical width does 16.6 %. `z` at 14.14× confirms the latent bottleneck is the hardest-working part of the model.

Attribution says the loss gradient does not flow much through a unit at the current optimum. It does **not** say the network computes the same function without it, and it is blind to redundancy in both directions: two duplicated units each look moderately used, while a unit that is dormant on average may be the one carrying rare line events. Concentration alone is not evidence either — a layer can be peaky and still carry its weight, which is why `load` rather than `top-20 %` drives the flag.
The only thing that settles a width question is an ablation: shrink the layer, retrain one seed, and re-score with `tools/acceptance_v0_8.py`.

Unlike the routing circuit, `save_full_circuit` needs real held-out events to measure, which a checkpoint alone does not carry. It re-runs line matching, gate-target construction, and the quantile/log10 transforms against the raw detector table you point it at, rebuilds the same train/test split (`random_state=42` by default, matching `tools/run_v0_8_real.py`), loads the model, and only then runs the attribution itself. Budget several minutes per source — 4 to 8

for the 3.4 M-event CR file on CPU, longer for a bigger one.
A practical note if you call this from a notebook: Python caches imported modules, so a kernel that imported `magi` before the package was last changed will keep regenerating the *old* widget, silently overwriting a newer file. Restart the kernel if the output does not match what you expect.

The stage layout (how many encoder/trunk/branch columns are drawn, and where they connect) is derived from the loaded checkpoint's own architecture, not hardcoded to any one source — it renders correctly whether the checkpoint has two particle types and four gate zones (Small/K-40) or six and six (CR).

Using these to check how the network works

- **Is the gate physically sane?** Figure 1: a type that cannot produce a given line should route essentially nothing to it. Lines appearing under `mu-` or a neutrino are a red flag.
- **Is the latent bottleneck saturated?** A high `load` on `z` says every latent dimension is working; combined with the KL discussion in Section 2, that is the case for *not* shrinking `latent_dim`.
- **Does one type use the network differently?** Switch buttons and watch which branch brightens. A type whose events are geometrically distinctive should light the position/direction branches more than one that is spectrally distinctive.
- **Is a branch carrying its weight?** Table 1, the `load` column — then an ablation, never the table alone.

5 The tools

Scripts in `tools/` operate around a run rather than inside one. All of them force CPU execution before importing `magi`. Run them from the repository root.

5.1 Training and scoring

`run_v0_8_real.py` A full training run on the real sources with live per-epoch logging and chunked generation, so a long run is observable and OOM-safe. Saves a checkpoint, spectra and a recovery report per source.

```
python tools/run_v0_8_real.py --sources CR Small --run-tag v0_8_2 \
    --epochs 40 --seed 42 --prior-zone-conditioning
```

Flags: `-sources`, `-run-tag`, `-epochs`, `-seed`, `-n-gen`, `-gen-chunk`, `-prior-zone-conditioning`, `-gate-target-bandwidth-mode` (default `resolution`), `-continuum-flow-bins`, `-warp-boost-cr`.

`acceptance_v0_8.py` The pass/fail harness. It grades a trained run against explicit thresholds, so an 80-minute run is scored objectively rather than by eye. The same numbers are the paper's validation table.

```
python tools/acceptance_v0_8.py --sources CR Small --seeds 42 7 13 \
    --json docs/_data/acceptance.json
```

It reports four things: per-line integral recovery (resolution-scaled $\pm 5\sigma$ window, local continuum subtracted, normalized for $N_{\mathrm{real}}/N_{\mathrm{gen}}$); near-line continuum ratio, i.e. whether the gate dug a hole beside each line; the energy-marginal Wasserstein distance, global and per decade band; and the coupling residuals.

The per-band Wasserstein distance on the CR source has a seed-to-seed σ/μ of 40–50 %, larger than most effects worth chasing. Two changes were adopted on single-seed evidence during development and both evaporated under a three-seed grid. Use `-seeds 42 7 13` and report mean ± std. The Small source is tighter, around 14 %.

`score_v0_7_2.py` Scores a v0.7.2 categorical-head run on the same axes, so the two heads can be compared like with like. It reads each checkpoint's own `preprocessing_metadata` rather than re-deriving it.

5.2 Audits

`line_centroid_audit.py` For each matched line, histograms the real spectrum locally at eV resolution, fits the centroid and reports |catalogue − measured| in FWHM units, failing loudly above one FWHM. Run this before committing to a long training run: a mispositioned line cannot be fixed by any amount of training.

`build_candidate_lines_from_geant4.py` Builds the candidate-line table from a GDML mass model and Geant4's fluorescence data. It takes transition *labels* from the Bearden table (only that directory carries them) and transition *energies* from the EADL table, because EADL is what the simulation actually emitted. Getting this backwards puts every instrumental line tens of eV off — up to 10 FWHM.

`gate_posterior_prior_audit.py` Localizes a per-line intensity failure to *where in the pipeline* it happens: the gate itself, or the latent distribution the gate is fed at generation time.

The gate is conditioned on z (`energy_flow_condition="z_cond"`). At training and reconstruction time it sees encoder codes, $z \sim q(z \mid x_{\text{real}})$; at generation time there is no real event, so it sees prior samples, $z \sim p(z \mid c)$. Section 2 argues those two differ substantially. This script measures whether that difference actually moves the gate's output, which is the step between "the latents differ" and "the line intensities are wrong".

It is training-free: it loads a checkpoint and runs two forward passes over the real dataset, one with encoder codes and one with prior samples. For every particle type × gate slot (continuum components plus each pinned line) it prints the mean gate probability under each, and their **prior/posterior ratio**:

```
python tools/gate_posterior_prior_audit.py --sources CR Small \
    --max-events-per-type 200000
```

```
type      slot                  post       prior   prior/post
gamma     continuum_0       9.3753e-01  9.4490e-01        1.008
gamma     Al K-alpha1       5.7883e-03  1.7573e-03        0.304
mu-       Cu K-alpha1       1.5495e-11  1.1889e-08      767.271
```

Read the ratio column, not the absolute fractions:

- ≈ 1 — the gate behaves the same whether it is fed encoder codes or prior samples. Any line error for that slot is the gate's own calibration, and gate-side fixes are the right place to look.
- $\ll 1$ — the line is routed *less* often at generation time than during training. The gamma rows above show this: Al Kα_1 at 0.304 means generation reaches that component roughly a third as often as reconstruction does, and no amount of gate reweighting during training will show up, because the training-time gate was already correct.

- $\gg 1$ — over-routing at generation time. The extreme values in the `mu-` rows are an artifact of dividing two numbers around 10^{-11}–10^{-8}: for a type that physically never produces that line, both fractions are numerically zero and the ratio is meaningless. Check the absolute columns before believing a large ratio.

A widespread departure from 1 is evidence that the *prior* is the thing to change — which is what motivated `prior_zone_conditioning` (§2) — rather than another round of gate reweighting, an experiment already run and recorded as a negative result in Section 8.

5.3 Synthetic stress tests

`synthetic_stress_test_coupled.py`, `_cr_multimodal.py` and `_small_singlepeak.py` build synthetic data with a known answer — an injected energy–geometry coupling, a rare-line cluster, a sparse tail. Use them to gate a change *before* spending a real run on it; they finish in minutes.

5.4 Unit tests

```
python -m pytest MAGI_package/tests/ -q
```

142 tests covering flow round-trip identity, checkpoint save/load config matching, gate-target construction (including the exact/resolution A/B), prior zone-conditioning, the two visualization tools (§4.10), and public API docstring coverage. They catch mechanical regressions; they do not tell you a trained model reproduced your source.

6 Example: a full run

This is a complete run on the reference cosmic-ray source, from raw data to a Geant4 source file. On a CPU-only Apple Silicon machine it takes roughly 90 minutes for 40 epochs plus generation.

6.1 The short path

Everything below is what `tools/run_v0_8_real.py` does. In practice:

```
export CUDA_VISIBLE_DEVICES=-1
python tools/run_v0_8_real.py --sources CR --run-tag my_run \
    --epochs 40 --seed 42 --prior-zone-conditioning \
    2>&1 | tee logs/my_run_CR.log
```

6.2 The long path, step by step

```
import os
os.environ["CUDA_VISIBLE_DEVICES"] = "-1"
import numpy as np, joblib, magi

magi.initialize_environment(seed=42, cpu_only=True)

# --- 1. load ------------------------------------------------------
df = magi.load_detector_table("TrainingData/alloutputDSCryoSphereCR.dat")
magi.report_basic_table_checks(df)

# --- 2. physical features -----------------------------------------
prep = magi.build_physical_features(
    df, center=(0.0, 0.0, -507.66), radius=100.0, source_type="cosmic_ray")
magi.print_physical_summary(prep)

# --- 3. lines: detect on the RAW energies, before the feature frame --
candidate_lines = magi.load_candidate_energy_lines(
```

```
    "CandidateLines/cryosphere_lines.json")["lines"]
E = prep["features"]["Energy"].to_numpy()

res = magi.detect_energy_lines(
    E, binning_mode="log_fixed_count", n_bins=1024,
    prominence_factor=3.0, window=5, candidate_lines=candidate_lines,
    refine_bin_width_mev=4.0e-6)
matched = [m for m in res["matched_lines"] if m["count"] >= 100]
matched += magi.confirm_unresolved_candidate_lines(
    E, candidate_lines, matched, resolution_ev=4.0)
magi.print_detected_energy_lines(res)

# --- 4. feature frame ------------------------------------------------
feature_pack = magi.build_feature_dataframe(
    prep, energy_binning_mode="log_fixed_count", n_bins=512, min_counts=20,
    geometry_transform="quantile_u_r_u_v_phi_r_phi_v",
    energy_transform="log10")          # the v0.8 head needs y = log10(E)
magi.report_feature_dataframe(feature_pack)

E_full = feature_pack["filtered_prep"]["features"]["Energy"].to_numpy()

gate_targets = magi.build_gate_targets(
    E_full, feature_pack["energy_bins"], matched,
    bandwidth_mode="resolution", bandwidth_fwhm_mev=4.0e-6)

line_positions_y = np.log10(
    [m["candidate_energy_mev"] for m in matched]).astype(np.float32)
line_logsigma = magi.line_logsigma_from_resolution(
    10.0 ** line_positions_y, resolution_ev=4.0, fwhm=True)

# --- 5. dataset: the gate targets ride along as extra feature columns -
feat = feature_pack["feat"].copy()
for j in range(gate_targets.shape[1]):
    feat[f"gate_target_{j}"] = gate_targets[:, j]
cont_cols = ("u_r_q", "u_v_q", "phi_r_q", "phi_v_q", "energy_y") + tuple(
    f"gate_target_{j}" for j in range(gate_targets.shape[1]))

dataset_pack   = magi.filter_particle_types_continuous_geometry(
                     feat=feat, prob_threshold=1e-5, cont_cols=cont_cols)
split_pack     = magi.split_feature_data(dataset_pack, random_state=42)
magi.report_split_summary(split_pack, dataset_pack["n_types"])
scaled_pack    = magi.scale_continuous_features(split_pack, scale_cols=())
condition_pack = magi.build_conditioning_and_weights(
                     scaled_pack, dataset_pack["n_types"],
                     idx_to_type=dataset_pack["idx_to_type"], alpha=0.5)
ds_pack        = magi.build_tf_datasets(condition_pack, batch_size=4096)
magi.report_tf_datasets(ds_pack)

# --- 6. per-type zone probabilities for the conditioned prior --------
n_zones   = gate_targets.shape[1]
zone_cols = dataset_pack["X_cont_raw"][:, -n_zones:]
zone_probs = np.zeros((dataset_pack["n_types"], n_zones))
for t in range(dataset_pack["n_types"]):
    mask = dataset_pack["y_type"] == t
    row  = zone_cols[mask].mean(axis=0) if mask.any() else np.zeros(n_zones)
    s    = row.sum()
    zone_probs[t] = row / s if s > 0 else np.eye(n_zones)[0]

# --- 7. model and training -------------------------------------------
model = magi.CVAE_MixEnergy_ContPhi_TaskAdaptive(
    n_types=dataset_pack["n_types"], latent_dim=8,
    line_positions_y=line_positions_y, line_logsigma_init=line_logsigma,
    continuum_mode="flow", continuum_flow_warp="cdf",
    energy_flow_condition="z_cond",
```

```
    prior="coupling", prior_zone_conditioning=True, zone_probs=zone_probs,
    gate_focal_gamma=1.0, w_gate_aux=2.0)

magi.compile_model(model, learning_rate=2e-4)
magi.print_model_structure(model)

history = magi.fit_model(
    model, ds_pack["train_ds"], ds_pack["val_ds"],
    epochs=40, callbacks=magi.build_default_callbacks(), verbose=2)

# --- 8. checkpoint ----------------------------------------------------
magi.save_final_trained_model(
    model=model, save_dir="trained_models/my_run", model_name="mix_CR",
    history=history, model_config=model.to_generation_config(),
    preprocessing_metadata=preprocessing_metadata,
    normalization_metadata=prep["normalization"])
```

6.3 Producing the Geant4 source file

```
python scripts/generate_geant_source.py \
    --save-dir trained_models/my_run \
    --model-name mix_CR \
    --metadata-file trained_models/my_run/mix_CR_metadata.json \
    --output-file generated/cr_source.dat \
    --n-events 10000000
```

The file format. One particle per line, whitespace-separated:

```
ParticleName Energy X Y Z Vx Vy Vz

gamma 0.131632939 -35.3167992 -93.4576950 -503.372406 \
      0.676084936 0.689519465 -0.259753942
e-    0.542123100  12.3456000  45.1234000 -510.332100 \
     -0.200000000 0.300000000 -0.932000000
```

Energy in MeV, position in mm, direction a normalized Cartesian unit vector. `ParticleName` must match a Geant4 particle definition exactly (`gamma`, `e-`, `proton`, `alpha`, `mu-`, `mu+`, ...) — an unrecognized name is the usual cause of "Unknown particle name" at load. The binary format (`-format binary`) carries the same fields.

What the Geant4 side must provide. Consuming this needs two classes in the application: a `PrimaryGeneratorAction` that can read the file and emit one primary per event instead of using `G4GeneralParticleSource`, and a `PrimaryGeneratorMessenger` exposing that as macro commands. In the companion project (`S1GDMLSRON_CryoSphereEmission`) the messenger declares thirteen commands in the `/generator/` directory:

Command	Purpose
Static file mode — read a pre-generated source	
`/generator/generatedFile`	Path to the source file. Loading it also *activates* file mode, so the next command is usually unnecessary.
`/generator/useGeneratedFile`	Enable/disable file mode explicitly.
`/generator/generatedFormat`	`text` or `binary`.
`/generator/binaryBufferSize`	Particles held in memory per read.
Auto mode — Geant4 invokes MAGI itself	
`/generator/autoGenerateMLInput`	Run the generator before loading.
`/generator/mlPython`	Python interpreter with `magi` installed.
`/generator/mlScript`	Path to `scripts/generate_geant_source.py`.
`/generator/mlModelDir`	Trained-run directory.
`/generator/mlModelName`	Checkpoint filename stem.
`/generator/mlMetadataFile`	The run's `*_metadata.json`.
`/generator/mlNumParticles`	Particles to generate.
`/generator/mlSeed`	Seed passed through to MAGI.
`/generator/mlChunkSize`	Generation chunk size.

In auto mode the generator shells out to exactly the command shown at the top of this subsection, assembled from those values:

```
"$mlPython" "$mlScript" --save-dir "$mlModelDir" \
    --model-name "$mlModelName" --metadata-file "$mlMetadataFile" \
    --output-file "$generatedFile" --n-events $mlNumParticles \
    --seed $mlSeed --chunk-size $mlChunkSize --format {text|binary}
```

so the C++ side needs no knowledge of MAGI beyond this argument list, and the same script is testable standalone from a shell.

A complete macro. This is the production configuration from the companion project's `try.mac`, trimmed to the generator part:

```
/run/initialize
/random/setSeeds 175 2024

/generator/autoGenerateMLInput true
/generator/generatedFormat     binary
/generator/binaryBufferSize    100000

/generator/mlPython   /path/to/envs/tf-metal/bin/python
/generator/mlScript   /path/to/MAGI/scripts/generate_geant_source.py
/generator/mlModelDir /path/to/MAGI/trained_models/my_run
/generator/mlModelName   mix_CR
/generator/mlMetadataFile /path/to/MAGI/trained_models/my_run/mix_CR_metadata.json

/generator/mlNumParticles 100000000
/generator/mlSeed         1752024
/generator/mlChunkSize    100000

/generator/generatedFile /path/to/tmp/geant_input_seed_1752024_CR.gntbin

/runAction/setTotalEvents 100000000
/run/beamOn               100000000
```

Give every job its own Geant4 seed (`/random/setSeeds`), its own MAGI seed (`/generator/mlSeed`) *and* its own `/generator/generatedFile` path. Sharing any of the three across concurrent jobs either duplicates the particle stream or has two processes writing one

> file. Prefer `binary` and auto-generation for large campaigns: particles are held in memory, so 10^8 in text form is heavy.

7 Example: a validation

Training finishing is not evidence that it worked. This is how a run gets graded.

7.1 The short path

```
python tools/acceptance_v0_8.py --sources CR --run-tag my_run \
    --seeds 42 7 13 --json docs/_data/my_run_acceptance.json
```

7.2 What it checks, and against what

Criterion	Threshold	Notes
Per-line recovery	[0.8, 1.2], two-sided	Only for lines above the detection floor. Two-sided because over-generation is the dominant failure.
Per-line stability	$\sigma \leq 0.15$ over ≥ 3 seeds	So a user's single unattended run likely lands in band.
Detection floor	$\geq 5\sigma$ Poisson	The particle-physics discovery convention. Below-floor lines are *not modelled*, and their absence is a correct result.
Near-line continuum	≥ 0.85	Did the gate dig a hole beside the line?
Energy marginal	Wasserstein ≤ 0.05	Global, on $\log_{10} E$.
Coupling	$\max \lvert\Delta\text{corr}\rvert \leq 0.05$	Over $(\log E, u_r, u_v, \phi_r, \phi_v)$.

7.3 Doing it by hand

```
real = magi.reconstruct_real_test_physics(
    split_pack["X_cont_test"], split_pack["E_test_raw"],
    center=(0.0, 0.0, -507.66), radius=100.0,
    geometry_metadata=geometry_metadata)

# Generate as many events as there are real ones - see the warning in 3.9.
gen_pack = magi.generate_latent_outputs(
    model, n_samples=real["E"].size, type_probs=type_probs,
    n_types=n_types, idx_to_type=idx_to_type)
reco = magi.reconstruct_generated_features(
    gen_pack, energy_head_mode="mixture",
    geometry_metadata=geometry_metadata, energy_metadata=energy_metadata)
gen  = magi.reconstruct_generated_physics(reco, center=(0,0,-507.66), radius=100.0)

# 1. marginals
scores = magi.compute_wasserstein_scores(real, gen)
print(scores["logE"])

# 2. geometric constraints that must hold by construction
magi.report_generated_constraints(gen, radius=100.0)
magi.report_final_ranges(real, gen)

# 3. per-line recovery
recovery = magi.compute_line_integral_recovery(
    real["E"], gen["E"], matched, energy_bins,
```

```
    energy_component_idx_gen=reco["energy_component_idx_gen"],
    resolution_ev=4.0, neighbour_lines=candidate_lines)
for r in recovery:
    print(f"{r['label']:20s} {r['recovery_ratio']:6.3f}  "
          f"sig={r['line_significance']:7.1f}")

# 4. the joint distribution - this is the one that matters
cols = ["logE", "u_r", "u_v", "phi_r", "phi_v"]
df_real, df_gen, df_both = magi.build_real_generated_featureframes(real, gen)
_, corr_real = magi.plot_correlation_matrix(df_real, cols, show=False)
_, corr_gen  = magi.plot_correlation_matrix(df_gen,  cols, show=False)
print("coupling residual:", np.abs(corr_gen - corr_real).to_numpy().max())

magi.compare_hist_with_residuals(real["E"], gen["E"], "Energy [MeV]",
                                 bins=200, ratio_clip=1.0)
```

7.4 Reading the output

- **Coupling residual** is the headline. Below 0.05 means the joint distribution is reproduced, which is what MAGI is for.
- **Wasserstein on** $\log_{10} E$ below 0.05 means the spectrum's shape is right.
- `line_significance` below 5 means the line is not significantly detected in the real data either; its recovery ratio is noise and should be ignored, not reported as a failure.
- `sideband_contaminated` true means a neighbouring line sits in the continuum side-bands, so the continuum subtraction for that line is unreliable.
- A statistical floor of $\sqrt{1/n_{\text{real}} + 1/n_{\text{gen}}}$ bounds how precisely any recovery ratio can be known. For a line with a few hundred events this is already several per cent.

8 Accuracy: what has actually been measured

All numbers are mean ± std over three seeds (42/7/13) on the two reference sources, produced by `tools/acceptance_v0_8.py`.

8.1 Validated

Quantity	CryoSphere-CR	CryoSphere-Small	Bar
Coupling, max $\lvert\Delta\text{corr}\rvert$	0.0285 ± 0.0101	0.0357 ± 0.0102	≤ 0.05
Wasserstein on $\log_{10} E$	0.0241 ± 0.0046	0.0065 ± 0.0006	≤ 0.05

The joint distribution holds up with error bars: every real cross-correlation over $(\log E, u_r, u_v, \phi_r, \phi_v)$ is reproduced inside the bar, on both sources, across seeds. The v0.7.2 categorical head does not achieve this — its coupling residual is 0.226, eight times worse.

8.2 Not validated

Per-line *intensities* are wrong by factors of roughly 0.7× to 4.9×, and unstable across seeds.

Line	Recovery	Status
CR Al Kα_1	0.959 ± 0.025	in band
CR Cu Kβ	1.034 ± 0.050	in band
CR e^+e^- 511 keV	1.825 ± 0.324	over-generated, unstable
CR Cu Kα_1	$2.052 \pm 0.034^\dagger$	over-generated
Small Al Kα_2	3.412 ± 0.376	far over
Small Cu Kα_1	$4.903 \pm 0.356^\dagger$	far over

† **These two are routing fractions, not window recoveries.** Cu Kα_1 and Cu Kα_2 are 21 eV apart, so each sits inside the other's continuum side-bands and the harness reports the window-based `recovery_ratio` as `n/a` (`sideband_contaminated`). The number quoted is the harness's `comp_rec`: the fraction of generated events the *gate* routed to that component, against the real fraction. It measures the same failure and is the only figure available for this line, but it is not the same quantity as the other rows and should not be compared against the $[0.8, 1.2]$ window bar directly.

Practical consequences:

- Line *positions* are reliable — pinned physical inputs, audited to ≤ 0.5 eV against the measured spectrum on all sources. It is the number of events in each line that is wrong.
- Do not use generated output to estimate a fluorescence or annihilation line flux, or any quantity dominated by one.
- The continuum between lines, and the total energy marginal, are within the bar.
- Rare lines fare worse than strong ones, and the failure is source-dependent. Measure on *your* source with the acceptance harness rather than assuming these numbers transfer.

8.3 Choosing between the heads

Neither head is correct for lines today. v0.7.2 has a better energy marginal on the cosmic-ray source (0.0141 versus 0.0241) but produces **zero** events in the 511 keV window against 52 225 real, and its coupling is eight times worse. If your use depends on correlations, use v0.8.2. If it depends only on the marginal spectrum and you want the most conservative option, v0.7.2 remains available.

8.4 Negative results already established

Recorded so they are not re-attempted without new information:

- **Per-line gate class weights**, calibrated from measured routing: no effect distinguishable from noise. The untouched control line moved more (+27 %) than either weighted line.
- **Continuum polish** (CDF-warp knot boost, more flow bins): a three-seed grid put every difference under 1.2σ of the noise.
- **Exact-match gate labels** (`bandwidth_mode="exact"`): physically the more faithful label, but it did not fix the line it was built for and broke a confirmed result and the coupling bar. See the warning in Section 4 and `docs/v0.8.1_line_truth.md` §14.2.

9 Limitations and traps

`build_physical_features`'s `center`/`radius` and the transforms in `core/geometry.py` all assume the surface events are scored on is a sphere. Adapting MAGI to a planar, cylindrical or box surface is **not** a matter of different parameters: it requires reworking the geometry-transform layer in both `preprocessing.py` and `generation/reconstruction.py`, which must stay in sync. This constrains the *scoring* surface, not the geometry you are simulating. The surface is an intermediate bookkeeping step in a multi-stage run, so the practical workaround for an awkward geometry is usually to place a dummy spherical tracking volume around the region of interest purely to give MAGI a surface to learn on, and leave the mass model itself alone.

The training data is a crossing spectrum, not deposited energy. Events are recorded where they cross a virtual sphere, with no detector response applied. Classical detector escape peaks may legitimately be absent; a null result there is a real result.

`geometry_transform` mismatches do not raise. They produce physically wrong output. The string is persisted in metadata for this reason — pass the metadata rather than re-specifying by hand.

Particle-type filtering silently drops rare species. If a rare-but-relevant type is missing from generated output, check `prob_threshold` first.

Energy binning limits which lines can be *found*. With the v0.8 mixture head, binning no longer limits where a line can be *placed*, but on a log grid spanning many decades a bin can be hundreds of eV wide, so close doublets fall in one bin. Use `refine_bin_width_mev` and `confirm_unresolved_candidate_lines`, then verify with `tools/line_centroid_audit.py`.

Quantile transforms need enough samples. Small datasets give unreliable tail behaviour in the fitted transform, which shows up as implausible extreme values in generated output.

Flux normalization depends on `source_type`. Choosing wrongly does not error. Radioactive sources additionally need the 13-column lineage schema.

Unit tests cover machinery, not physics. 142 passing tests do not tell you a trained model reproduced your source. That is what Section 7 is for.

CPU-only by default. On this workload `tensorflow-metal` is slower than the CPU.

10 Where to look next

Document	Contents
`MAGI_package/docs/USAGE.md`	Short-form version of this manual.
`Example_Usage.ipynb`	Runnable walkthrough on synthetic data.
`docs/v0.8.1_line_truth.md`	The measurement record behind Section 8: what was tried, what it showed.
`docs/v0.8.2_RoadmapForAdoption.md`	The remaining plan and the acceptance bar as re-stated.
`docs/v0.8_learnable_prior_theory.md`	Why the conditional prior was needed.
`docs/v0.8_v072_comparison.md`	Head-to-head history. Note its absolute numbers predate the metric fix.

Include the run directory's `*_config.json` and `*_metadata.json`, the acceptance-harness JSON if you have it, and the seed. Most surprising results trace back to a preprocessing contract mismatch that those files make visible immediately.

References

[1] Diederik P. Kingma and Max Welling. Auto-Encoding Variational Bayes. In *International Conference on Learning Representations (ICLR)*, 2014.

[2] Kihyuk Sohn, Honglak Lee, and Xinchen Yan. Learning Structured Output Representation using Deep Conditional Generative Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28, 2015.

[3] S. Agostinelli et al. Geant4—a simulation toolkit. *Nuclear Instruments and Methods in Physics Research A*, 506(3):250–303, 2003.

[4] J. Allison et al. Recent developments in Geant4. *Nuclear Instruments and Methods in Physics Research A*, 835:186–225, 2016.

[5] George Papamakarios, Eric Nalisnick, Danilo J. Rezende, Shakir Mohamed, and Balaji Lakshminarayanan. Normalizing Flows for Probabilistic Modeling and Inference. *Journal of Machine Learning Research*, 22(57):1–64, 2021.

[6] Ivan Kobyzev, Simon J.D. Prince, and Marcus A. Brubaker. Normalizing Flows: An Introduction and Review of Current Methods. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(11):3964–3979, 2021.

[7] Conor Durkan, Artur Bekasov, Iain Murray, and George Papamakarios. Neural Spline Flows. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019. Monotonic rational-quadratic spline transforms; the continuum density in magi/core/flows.py.

[8] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal Loss for Dense Object Detection. In *IEEE International Conference on Computer Vision (ICCV)*, 2017. Focal down-weighting of easy examples; the gate loss must route lines that are 1e-5 of the sample.

[9] Jakub M. Tomczak and Max Welling. VAE with a VampPrior. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2018. The aggregated-posterior/prior mismatch argument for replacing N(0,I) with a learned prior.

[10] Laurent Dinh, Jascha Sohl-Dickstein, and Samy Bengio. Density Estimation using Real NVP. In *International Conference on Learning Representations (ICLR)*, 2017. Affine coupling layers; the architectural basis of the conditional prior in magi/core/priors.py.

[11] J. A. Bearden. X-Ray Wavelengths. *Reviews of Modern Physics*, 39(1):78–124, 1967. Source of the transition labels in the candidate-line table.

[12] S. T. Perkins, D. E. Cullen, M. H. Chen, J. H. Hubbell, J. Rathkopf, and J. Scofield. Tables and Graphs of Atomic Subshell and Relaxation Data Derived from the LLNL Evaluated Atomic Data Library (EADL), Z = 1–100. Technical Report UCRL-50400, Vol. 30, Lawrence Livermore National Laboratory, 1991. The fluorescence energies Geant4 actually emits, and therefore the ones MAGI pins its line components at.

[13] Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations (ICLR)*, 2015. Default optimizer in magi.compile_model.

[14] N. S. Schmidt, O. I. Abbate, Z. M. Prieto, J. I. Robledo, J. I. Márquez Damián, A. A. Márquez, and J. Dawidowski. KDSource, a tool for the generation of Monte Carlo particle sources using kernel density estimation. *Annals of Nuclear Energy*, 177:109309, 2022.

[15] Baran Hashemi and Claudius Krause. Deep generative models for detector signature simulation: A taxonomic review. *Reviews in Physics*, 12:100092, 2024.

[16] Michela Paganini, Luke de Oliveira, and Benjamin Nachman. CaloGAN: Simulating 3D high energy particle showers in multilayer electromagnetic calorimeters with generative adversarial networks. *Physical Review D*, 97(1):014021, 2018.

[17] Claudius Krause and David Shih.

[18] ATLAS Collaboration. Deep generative models for fast photon shower simulation in ATLAS. *Computing and Software for Big Science*, 8:7, 2024.

[19] S. Lotti et al. Review of the Particle Background of the Athena X-IFU Instrument. *The Astrophysical Journal*, 909(2):111, 2021.

[20] Mukund Sundararajan, Ankur Taly, and Qiqi Yan. Axiomatic Attribution for Deep Networks. In *International Conference on Machine Learning (ICML)*, volume 70, pages 3319–3328, 2017. Integrated Gradients; the input-feature attribution that conductance generalizes to internal units.

[21] Kedar Dhamdhere, Mukund Sundararajan, and Qiqi Yan. How Important Is a Neuron? In *International Conference on Learning Representations (ICLR)*, 2019. Conductance. The usage metric behind magi.save_full_circuit.

[22] Gabriel Erion, Joseph D. Janizek, Pascal Sturmfels, Scott M. Lundberg, and Su-In Lee.